

CS141 Coursework 2 – LGPT

For my program, I tried to as much as possible separate each part of the coursework specification into its own section, to keep a good separation of concerns in my program. I chose to put most of the code relating to evaluations in its own file entirely (Calculator.hs) as there were many helper functions for this part of the code that were not needed for the rest of the code so made sense to go in their own file and import the functions needed into the main TUI.hs file. I also did the same for the optional part of the coursework where I made a file just for functions relating to football requests. I also allowed the evaluations part of the program to handle variables as an optional extension

Also when using Monads in the program I decided that using MTL (for the most part) would be the best method as they are more flexible and also avoid the need to manually lift between layers of the stack and although most of my stacks were small this was still more convenient in my opinion. This decision was informed by information in the lecture and also by Yao [1] .

For the date requests part, I chose to use the Data.Time library to help as this library contained many useful functions for working with dates. To find the current time I chose to use getZonedTime and then convert this into a local time so that the date will be exactly correct and accounts for British Summer Time. To parse a date I chose to use the inbuilt Data.Time parser as this handles all the little edge cases in validating a date (such as handling leap years) which would be very repetitive to implement myself.

For the expressions request part, I chose to store expressions as an initial value and then a list of partial operators to apply to the initial value in turn. This is because the precedence required was just left to right as the operators appeared so if I stored the operators in a list I could use foldl' to apply these partial operators to the initial value in turn. When parsing expressions I chose to use Text.Megaparsec.Char.Lexer to avoid the need to manually handle white space in the expression. I chose to, as recommend by the library, assume all symbols had no whitespace before them and to consume all whitespace after a symbol when consuming said symbol. To do this I used the lexeme function.

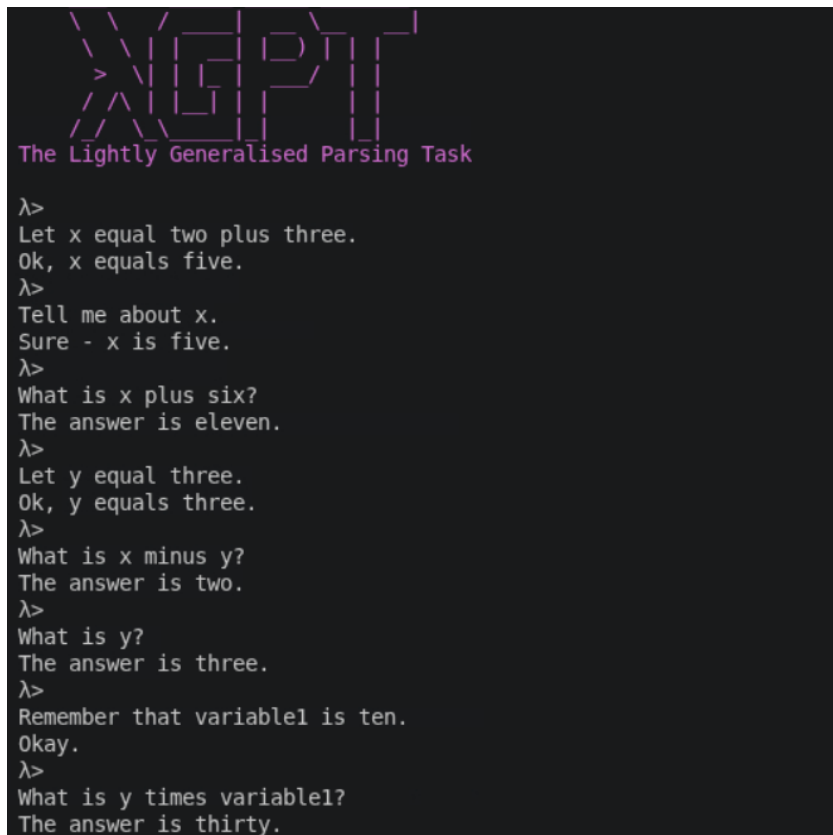
For the remembering parts, I chose to combine both IO and State monads together using the MTL (justified earlier). Then when actually running the REPL I used a concrete StateT transformer which I then unwrapped using evalStateT so that type of the main runREPL stayed the same. Inside the state I chose to store the mapping of names to things using a Map from String to String. This allowed for efficient look up of names compared to the use of a list for example. When parsing a name or thing I chose to use someTill or takeWhile1P depending on whether the delimiter after the name/thing was a collection of words or a single character. This method of parsing was required as names/things could be multiple words so I couldn't just parse till the next space. I chose to have this variation as takeWhile1P is more efficient so should be used where possible. For the use of 'that' in expressions I knew that I wanted to implement variables in the expression as part of my optional task so I simply chose to implement variables first then store 'that' as a variable and update it after each evaluation.

For the first part of my optional section, I chose to implement the use of variables within the expressions. I chose to do this as the use of variables fit quite nicely with the already existing memory feature and seemed like a useful addition to the chatbot. A variable can be assigned to an expression (which can also be a single value). A variable can be any singular word. This is possible by the parsing of variable being similar to the one used for name. Assigning of a variable is done using the command “Let [variable] equal [expression]”. Variables are stored in the same Map as name/thing so by extension can also use any of the chatbot features related to name/thing (however if you try to use a name that doesn’t store a number in an expression the program will return a useful error stating so, see figure 2). Variables store their value as the longhand string of the evaluated evaluation they represent. This is so they can be stored in the Map from String to String. This does lead to converting the Variable to a string to store it and then back to a number to evaluate it however I’d argue the combination of storing name things with variable values is a useful trade off to have as it simplifies the storage of the variables at the slight expense of execution speeds. An example of the use of variables is shown in figure 1.

For the second part of my optional section, I chose to implement a football chatbot that can tell you the next fixture of any current Premier League football club and also what position they are in the table. To do this I made API requests to a website called <http://api.football-data.org/v4> which is an API which stores lots of information about football fixtures and teams and with the free tier you can get information relating to the Premier League. The fundamentals of my understanding of API requests in Haskell came from Sherman [2] as well as the lecture that covered http requests. To make API requests I used the http-conduit library to make the requests as well as the aeson library to handle the JSON objects. The aeson library is used for the FromJSON class which is used to unpack the different parts of the JSON object returned from the request. To get the next fixture of a team the request : “Who is [team] playing next?” should be used and to get the position in the league of the team you should: “What position are [team] in the Premier League?”. Examples of this are in figure 3 and the possible teams that can be used are: Chelsea, Arsenal, Aston Villa, Bournemouth, Brentford, Brighton & Hove Albion, Burnley, Crystal Palace, Everton, Fulham, Leeds United, Liverpool, Manchester City, Manchester United, Newcastle, Nottingham Forest, Sunderland, Tottenham, West ham and Wolves. To test these are correct simply google the teams next premier league fixture and compare. Note that the chatbot only shows Premier League games and not any other competitions (such as the FA cup) as this is all the free API gives access to relating to English clubs. When displaying who a team is playing, the fixture will be displayed with the home team first and then the away team regardless of which team was queried about and this is intentional as when listing fixtures in football it is convention to list the home team first. To handle the API key securely I used the dotenv library to load a .env file which stored the API key. I then loaded the API key from the environment variable using System.Environment. This was done following the advice of Alex Dixon.

Figures:

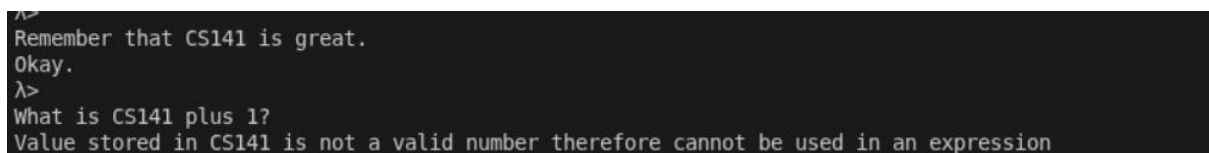
Figure 1:



```
XGPT
The Lightly Generalised Parsing Task

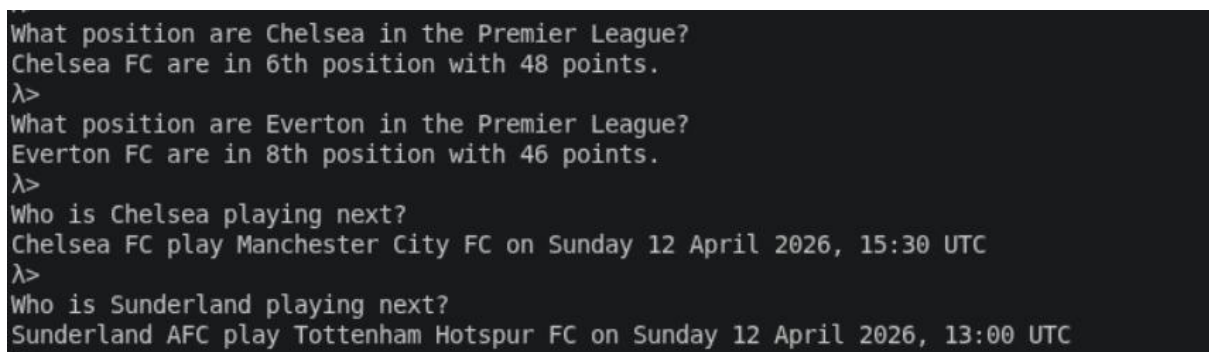
λ>
Let x equal two plus three.
Ok, x equals five.
λ>
Tell me about x.
Sure - x is five.
λ>
What is x plus six?
The answer is eleven.
λ>
Let y equal three.
Ok, y equals three.
λ>
What is x minus y?
The answer is two.
λ>
What is y?
The answer is three.
λ>
Remember that variable1 is ten.
Okay.
λ>
What is y times variable1?
The answer is thirty.
```

Figure 2:



```
λ>
Remember that CS141 is great.
Okay.
λ>
What is CS141 plus 1?
Value stored in CS141 is not a valid number therefore cannot be used in an expression
```

Figure 3:



```
λ>
What position are Chelsea in the Premier League?
Chelsea FC are in 6th position with 48 points.
λ>
What position are Everton in the Premier League?
Everton FC are in 8th position with 46 points.
λ>
Who is Chelsea playing next?
Chelsea FC play Manchester City FC on Sunday 12 April 2026, 15:30 UTC
λ>
Who is Sunderland playing next?
Sunderland AFC play Tottenham Hotspur FC on Sunday 12 April 2026, 13:00 UTC
λ>
```

References:

1. Yao W. Monad Transformers 101 [Internet]. williamyoah.com; 2023 Jun 10 [cited 2026 Apr 2]. Available from: <https://williamyoah.com/posts/2023-06-10-monad-transformers-101.html>
2. Sherman B. Consuming JSON API Data with Haskell [Internet]. bitbybrad.com; 2021 Feb 9 [cited 2026 Apr 2]. Available from: <https://bitbybrad.com/2021-02-09-json-api-haskell/>